



FOOD STORE

Sistema de Gestión de Pedidos de Comida

Especificación Técnica del Sistema · Versión 4.0 · Spec-Driven Development

Campo	Valor
Materia	Programación 4
Carrera	Tecnicatura Universitaria en Programación (TUP)
Modalidad	Trabajo Práctico Integrador (TPI)
Stack	React + TypeScript + FastAPI + PostgreSQL
Metodología	Spec-Driven Development (SDD)
Versión doc.	3.0 — incorpora correcciones de auditoría + UoW + MercadoPago + Zustand completo

1. Visión General del Sistema

Food Store es una aplicación web full-stack para la gestión integral de un negocio de comidas. Permite a los clientes explorar el catálogo, agregar productos al carrito, realizar pedidos con pago integrado vía MercadoPago y hacer seguimiento en tiempo real del estado de su pedido. Los administradores gestionan el catálogo, el stock, los pedidos y los usuarios desde un panel centralizado.

1.1 Objetivos del Sistema

#	Actor	Objetivo principal
OBJ-01	Cliente	Navegar el catálogo, gestionar carrito, pagar con MercadoPago y rastrear pedidos con trazabilidad completa
OBJ-02	Administrador	Gestionar categorías, productos, stock y ciclo de vida de pedidos desde el panel
OBJ-03	Gestor de Stock	Controlar disponibilidad y cantidad de stock de productos
OBJ-04	Gestor de Pedidos	Avanzar el estado de los pedidos según la máquina de estados definida
OBJ-05	Sistema	Garantizar trazabilidad completa de transiciones de estado mediante audit trail append-only
OBJ-06	Sistema	Procesar y registrar pagos a través de la pasarela MercadoPago de forma atómica

1.2 Alcance v3.0

- Autenticación y autorización con JWT y RBAC (4 roles) + invalidación de refresh token en base de datos
- Catálogo de productos con categorías jerárquicas e ingredientes con campo es_alergeno
- Carrito de compras con persistencia mediante Zustand + localStorage
- Gestión de pedidos con máquina de estados de 6 estados y audit trail append-only
- Pasarela de pagos MercadoPago Checkout API: tarjeta de crédito/débito, Rapipago, Pago Fácil, Cuenta MP
- Notificaciones webhook IPN de MercadoPago para confirmación automática de pagos
- Módulo DireccionEntrega: CRUD completo con dirección principal por usuario
- Catálogo FormaPago con seed data: MERCADOPAGO, EFECTIVO, TRANSFERENCIA
- Panel de administración completo: dashboard con recharts, CRUD de entidades, gestión de pedidos y stock
- Rate limiting con slowapi: máximo 5 intentos fallidos por IP en 15 minutos en el endpoint de login
- CORS configurado correctamente con CORSMiddleware para la separación frontend/backend

- Seed data obligatorio: roles, estados de pedido, formas de pago y usuario administrador por defecto
- API REST completamente documentada con FastAPI/OpenAPI — accesible en /docs y /redoc
- Diseño responsive mobile-first con Tailwind CSS

1.3 Stack Tecnológico

Capa	Tecnología	Versión	Rol en el sistema
Frontend	React + TypeScript	18.x + 5.x	UI, enrutamiento, componentes
Frontend	Vite	5.x	Build tool y dev server
Frontend	Tailwind CSS	3.x	Estilos utility-first
Frontend	TanStack Query	5.x	Fetching, caché y sincronización de datos del servidor
Frontend	TanStack Form	0.x	Gestión de formularios con validación
Frontend	Zustand	4.x	Estado global del cliente (carrito, sesión, pagos, UI)
Frontend	Axios	1.x	Cliente HTTP con interceptors JWT
Frontend	recharts	2.x	Gráficos del dashboard de administración
Frontend	@mercadopago/sdk-react	—	SDK oficial MercadoPago para tokenización PCI-compliant
Backend	FastAPI	0.111+	Framework REST + generación automática OpenAPI
Backend	SQLModel	0.0.19+	ORM + schemas Pydantic integrados
Backend	PostgreSQL	15+	Base de datos relacional
Backend	Alembic	1.13+	Migraciones versionadas de base de datos
Backend	PyJWT / python-jose	—	Generación y validación de JWT
Backend	Passlib (bcrypt)	—	Hashing de contraseñas (cost factor ≥ 12)
Backend	mercadopago	2.3.0+	SDK oficial MercadoPago Python
Backend	slowapi	0.1.9+	Rate limiting por IP en endpoints críticos

2. Arquitectura del Sistema

2.1 Capas del backend

El backend aplica una arquitectura de capas con módulos por feature. En v3.0 se incorpora el patrón Unit of Work (UoW) entre la capa de servicio y los repositorios, garantizando atomicidad transaccional. El flujo de dependencias es unidireccional y no puede invertirse.

Capa	Archivo de referencia	Responsabilidad	¿Conoce a?
Router	router.py	HTTP puro: parsear request, validar esquema Pydantic, delegar al Service, serializar response con response_model. No contiene lógica de negocio.	Service
Service	service.py	Lógica de negocio: stateless, orquesta operaciones sobre los repositorios a través del UoW. Lanza HTTPException con códigos semánticos. No hace commit/rollback directamente.	UoW
Unit of Work	core/uow.py	Gestión de transacción: abre la sesión de BD, provee acceso a todos los repositorios, hace commit() automático al salir sin excepciones o rollback() si ocurre algún error.	Repository, Session
Repository	repository.py	Acceso a BD: queries sin lógica de negocio. Hereda de BaseRepository[T] genérico. Recibe la sesión del UoW por inyección.	Model, Session
Model	model.py	SQLModel tables + relaciones. Sin imports de capas superiores. Define la estructura de la base de datos.	Ninguna

Regla de oro — flujo de imports

Router → Service → UoW → Repository → Model

Ninguna capa puede importar de la capa superior. Un Model nunca importa de un Service. Un Repository nunca importa de un Router.

2.2 Capas del frontend

Capa	Directorio	Responsabilidad
Page	pages/	Solo define la ruta. Delega completamente a los componentes de features. Sin lógica propia.

Capa	Directorio	Responsabilidad
Feature	features/	Componentes de dominio de negocio: formularios, tablas, modales. Usan hooks propios.
Component	components/	Primitivos reutilizables sin lógica de negocio (Button, Input, Badge, Modal, Skeleton).
Hook	hooks/	Encapsulan lógica de TanStack Query (useQuery / useMutation). Un hook por dominio.
Store	store/	Zustand: estado global del cliente. Separado del estado del servidor (TanStack Query).
API	api/	Funciones Axios puras por dominio. Sin estado, sin hooks. Solo llamadas HTTP.
Types	types/	Interfaces y tipos TypeScript globales. Sin imports de otras capas.

Separación Zustand / TanStack Query

Zustand gestiona el estado del CLIENTE: carrito, sesión, proceso de pago, UI local (modales, sidebar).

TanStack Query gestiona el estado del SERVIDOR: productos, pedidos, dashboard. Datos remotos con caché automático.

Mezclar ambos tipos de estado en el mismo store es un error arquitectónico que debe evitarse.

2.3 Estructura de directorios — Backend v3.0

Ruta	Descripción
app/main.py	Crea la app FastAPI. Registra CORSMiddleware, rate limiter y todos los routers.
app/core/config.py	Settings con pydantic-settings. Lee todas las variables de entorno desde .env.
app/core/security.py	Funciones de bcrypt hash y JWT encode/decode. Sin estado.
app/core/dependencies.py	Dependencias de FastAPI: get_current_user(), require_role(). Se inyectan en routers.
app/core/uow.py	★ NUEVO — Unit of Work. Gestiona ciclo de vida de sesión SQLAlchemy. Provee repositorios.
app/core/base_repository.py	★ NUEVO — BaseRepository[T] genérico con CRUD básico. Todos los repos heredan de aquí.
app/core/exceptions.py	Handlers HTTP globales para 404, 422, 500. Nunca expone traceback.
app/db/base.py	SQLModel engine + create_all(). Pool de conexiones.
app/db/session.py	get_session() — generador de sesión para inyección de dependencias.
app/db/seed.py	★ NUEVO — Inserta datos iniciales: roles, estados, formas de pago, admin por defecto.

Ruta	Descripción
app/modules/auth/	Login, registro, refresh, logout. JWT access (30 min) + refresh (7 días).
app/modules/refreshTokens/	★ NUEVO — Modelo RefreshToken en BD para invalidación segura en logout.
app/modules/usuarios/	CRUD usuarios + asignación de roles RBAC. Soft delete.
app/modules/direcciones/	★ NUEVO — CRUD completo DireccionEntrega por usuario.
app/modules/categorias/	Categorías jerárquicas con CTE recursiva. Soft delete con validación de dependencias.
app/modules/productos/	Catálogo con Ingrediente completo (es_alergeno). Stock como campos en Producto.
app/modules/pedidos/	Dominio central: máquina de estados, audit trail, historial append-only.
app/modules/pagos/	★ NUEVO — Integración MercadoPago: crear pago, webhook IPN, registro de transacciones.
app/modules/admin/	Dashboard con métricas, gestión de stock y usuarios desde el panel.

2.4 Estructura de directorios — Frontend v3.0

Ruta	Descripción
src/main.tsx	Entry point. Monta providers: QueryClientProvider, routers.
src/App.tsx	Router principal. Agrupa rutas públicas, de cliente y de admin.
src/api/axiosClient.ts	Instancia base Axios. baseURL = VITE_API_URL + /api/v1. Interceptors JWT y refresh automático.
src/api/authApi.ts	login(), register(), refresh(), logout(), me().
src/api/productosApi.ts	list(), getById(), toggleDisponibilidad().
src/api/pedidosApi.ts	list(), getById(), crear(), avanzarEstado(), historial().
src/api/pagosApi.ts	★ NUEVO — crearPago(), getPago().
src/api/adminApi.ts	getDashboard(), getStock(), updateStock().
src/store/authStore.ts	★ AMPLIADO — accessToken, usuario, isAuthenticated, hasRole(), login(), logout(), refreshToken().
src/store/cartStore.ts	★ AMPLIADO — items, addItem(), removeItem(), updateCantidad(), itemCount(), subtotal(), costoEnvio(), total().
src/store/paymentStore.ts	★ NUEVO — status del pago MP en curso, mpPaymentId, statusDetail, reset().
src/store/uiStore.ts	★ NUEVO — cartOpen, sidebarOpen, confirmModal. Centraliza el estado de UI.
src/hooks/useAuth.ts	useLogin, useRegister, useMe.
src/hooks/useProductos.ts	useProductos, useProducto, useToggleDisponibilidad.

Ruta	Descripción
src/hooks/usePedidos.ts	usePedidos (polling 30s), usePedido, useAvanzarEstado, useHistorial.
src/hooks/usePagos.ts	★ NUEVO — useCrearPago, usePago.
src/hooks/useAdmin.ts	useDashboard, useStock, useUpdateStock.
src/features/auth/	LoginForm, RegisterForm, ProtectedRoute HOC.
src/features/store/	CatalogoGrid, ProductoDetail, CartDrawer, CheckoutForm (con CardPayment de MP).
src/features/pedidos/	PedidosList, PedidoDetail, HistorialTimeline, PaymentStatus.
src/features/admin/	Dashboard, CRUD de categorías/productos/pedidos/usuarios, StockTable.
src/types/index.ts	Interfaces globales: Producto, Pedido, Usuario, Pago, CartItem, FormaPago.

3. Modelo de Datos

El esquema aplica Tercera Forma Normal (3FN), Soft Delete (deleted_at TIMESTAMPTZ), Snapshot Pattern en pedidos y Audit Trail append-only en HistorialEstadoPedido. Las correcciones de auditoría incorporan las entidades faltantes en v2.0.

3.1 Dominio 1 — Identidad y Acceso

Entidad	Campo clave	Tipo	Restricción	Notas
Usuario	id	BIGSERIAL	PK	Soft-delete vía deleted_at
Usuario	email	VARCHAR(254)	UQ, NN	Validar formato EmailStr
Usuario	password_hash	CHAR(60)	NN	bcrypt. Nunca almacenar plaintext
Rol	codigo	VARCHAR(20)	PK	Catálogo: ADMIN, STOCK, PEDIDOS, CLIENT
UsuarioRol	(usuario_id, rol_id)	BIGINT×2	PK compuesta	Pivot N:M. Incluye asignado_por_id
RefreshToken ★	token_hash	CHAR(64)	UQ, NN	SHA-256 del token. revoked_at NULL = activo
RefreshToken ★	expires_at	TIMESTAMPTZ	NN	
RefreshToken ★	revoked_at	TIMESTAMPTZ	NULL	Se completa en logout
DireccionEntrega ★	alias	VARCHAR(50)	NULL	Ej: 'Casa', 'Trabajo'
DireccionEntrega ★	calle + numero	VARCHAR	NN	
DireccionEntrega ★	es_principal	BOOLEAN	NN, default false	Solo una por usuario

3.2 Dominio 2 — Catálogo de Productos

Entidad	Campo clave	Tipo	Restricción	Notas
Categoria	parent_id	BIGINT	FK self-ref, NULL	Jerarquía recursiva. ON DELETE SET NULL
Producto	precio_base	DECIMAL(10, 2)	CHECK ≥ 0, NN	Snapshot al crear pedido
Producto ★	stock_cantidad	INTEGER	CHECK ≥ 0, NN, default 0	Gestionado por rol STOCK

Entidad	Campo clave	Tipo	Restricción	Notas
Producto ★	disponible	BOOLEAN	NN, default true	Toggle manual independiente del stock
Ingrediente ★	nombre	VARCHAR(100)	UQ, NN	Especificación completa en v3.0
Ingrediente ★	es_alergeno	BOOLEAN	NN, default false	Para badge de alérgenos en UI
ProductoCategoria	(producto_id, cat_id)	BIGINT×2	PK compuesta	Pivot N:M
ProductoIngrediente	es_removible	BOOLEAN	NN	Habilita personalización de pedido
FormaPago ★	codigo	VARCHAR(20)	PK	Catálogo. Ver seed data.
FormaPago ★	habilitado	BOOLEAN	NN, default true	Se puede deshabilitar sin eliminar

3.3 Dominio 3 — Ventas, Pagos y Trazabilidad

Entidad	Campo clave	Tipo	Restricción	Notas
EstadoPedido	codigo	VARCHAR(20)	PK	Catálogo. Ver máquina de estados.
EstadoPedido	es_terminal	BOOLEAN	NN	true = no admite transiciones salientes
Pedido	estado_codigo	VARCHAR(20)	FK → EstadoPedido	Estado actual del pedido
Pedido	total	DECIMAL(10, 2)	CHECK ≥ 0, NN	Snapshot inmutable al crear
Pedido	costo_envio	DECIMAL(10, 2)	NN, default 50.00	Valor fijo v1. Documentado.
Pedido	descuento	DECIMAL(10, 2)	NN, default 0	Reservado. Sin lógica de cupones en v1.
Pedido	forma_pago_codigo	VARCHAR(20)	FK → FormaPago	
Pedido	direccion_id	BIGINT	FK → Direccion, NULL	NULL = retiro en local
DetallePedido	nombre_snapshot	VARCHAR(200)	NN	Snapshot: nombre del producto al crear
DetallePedido	precio_snapshot	DECIMAL(10, 2)	NN	Snapshot: precio al crear. Inmutable.

Entidad	Campo clave	Tipo	Restricción	Notas
DetallePedido	personalizacion	BIGINT[]	NULL	IDs de ingredientes removidos
HistorialEstadoPedido	estado_desde	VARCHAR(20)	FK, NULL	NULL = transición inicial (RN-02)
HistorialEstadoPedido	estado_hacia	VARCHAR(20)	FK, NN	ON DELETE RESTRICT
HistorialEstadoPedido	created_at	TIMESTAMP TZ	NN	Nunca updated_at. Append-only (RN-03).
Pago ★	mp_payment_id	BIGINT	UQ, NULL	ID devuelto por MercadoPago
Pago ★	mp_status	VARCHAR(30)	NN	pending / approved / rejected
Pago ★	mp_status_detail	VARCHAR(100)	NULL	accredited, cc_rejected_other_reason...
Pago ★	external_reference	VARCHAR(100)	UQ, NN	ID del Pedido como referencia MP
Pago ★	idempotency_key	VARCHAR(100)	UQ, NN	UUID generado por el backend. Evita cobros duplicados.

3.4 Máquina de estados — Pedido

La entidad EstadoPedido es un catálogo. La capa de servicio valida la transición contra el mapa definido antes de cada INSERT en HistorialEstadoPedido. Ningún router puede saltar esta validación.

Código	Descripción	Orden	es_terminal	Transiciones válidas
PENDIENTE	Pedido creado, pago pendiente	1	false □	→ CONFIRMADO, → CANCELADO
CONFIRMADO	Pago procesado y confirmado	2	false □	→ EN_PREP, → CANCELADO
EN_PREP	En preparación en cocina	3	false □	→ EN_CAMINO, → CANCELADO (solo operador)
EN_CAMINO	Despachado al cliente	4	false □	→ ENTREGADO
ENTREGADO	Entrega confirmada	5	true ✓	— (estado terminal)
CANCELADO	Pedido cancelado	6	true ✓	— (estado terminal)

Reglas de negocio — Pedidos

RN-01: Un estado con `es_terminal = true` no admite transiciones salientes. La validación ocurre en la capa de servicio, nunca en el router.

RN-02: El primer registro de `HistorialEstadoPedido` siempre tiene `estado_desde = NULL` (transición inicial).

RN-03: La tabla `HistorialEstadoPedido` es `append-only`. Ninguna capa puede emitir `UPDATE` ni `DELETE` sobre ella.

RN-04: El total, nombre y precio en `DetallePedido` son un snapshot inmutable capturado al crear el pedido.

4. Autenticación y Autorización

4.1 Flujo de autenticación

Pas o	Actor	Acción	Resultado esperado
1	Cliente	POST /api/v1/auth/login con email + password	HTTP 200 + access token (30 min) + refresh token (7 días)
2	Frontend	Almacena access token en authStore (Zustand). NO en localStorage directamente.	Token disponible para interceptor Axios
3	Frontend	Interceptor Axios agrega Authorization: Bearer <token> a cada request	Request autenticado hacia el backend
4	Backend	Dependency get_current_user() valida JWT y carga el usuario	Objeto usuario inyectado en el handler
5	Backend	require_role([Rol.ADMIN]) verifica roles del token	HTTP 403 si rol insuficiente
6	Cliente	POST /api/v1/auth/refresh con refresh token (cookie httpOnly)	Nuevo access token sin requerir re-login
7	Cliente	POST /api/v1/auth/logout	Refresh token marcado como revoked_at en tabla RefreshToken

4.2 Roles y permisos

Rol	Código	Permisos principales	Restricciones
Administrador	ADMIN	CRUD completo: usuarios, categorías, productos, pedidos, stock. Asigna roles.	Sin restricciones.
Gestor de Stock	STOCK	Leer productos, actualizar stock_cantidad y disponible, ver ingredientes.	Sin acceso a usuarios ni datos financieros.
Gestor de Pedidos	PEDIDOS	Ver todos los pedidos, avanzar estados CONFIRMADO → ENTREGADO, ver historial.	Sin acceso a productos ni finanzas.
Cliente	CLIENT	Ver catálogo, gestionar carrito, crear pedidos, ver sus propios pedidos.	Solo accede a sus propios datos.

4.3 Payload del access token (JWT)

Claim	Tipo	Descripción
sub	string	usuario_id como string
email	string	Email del usuario
roles	string[]	Lista de códigos de roles: ['CLIENT'], ['ADMIN'], etc.

Claim	Tipo	Descripción
iat	integer	Timestamp de emisión (issued at)
exp	integer	Timestamp de expiración. 30 minutos desde iat.

4.4 Rate limiting en autenticación

Configuración	Valor
Librería	slowapi — integración nativa con FastAPI
Límite	5 intentos fallidos por dirección IP en 15 minutos
Endpoint protegido	POST /api/v1/auth/login
Respuesta al superar	HTTP 429 Too Many Requests con header Retry-After
Configuración	Middleware global en app/main.py + decorador @limiter.limit() en el router de auth

5. Especificación de API REST

Todos los endpoints usan el prefijo `/api/v1`. Los errores siguen RFC 7807 (Problem Details). La documentación interactiva se genera automáticamente en `/docs` (Swagger UI) y `/redoc`.

Convenciones globales

Error estándar: `{ "detail": "mensaje", "code": "ERROR_CODE", "field": "campo_opcional" }`

Paginación: `GET /recursos?page=1&size=20` → `{ "items": [...], "total": N, "page": 1, "size": 20, "pages": P }`

Soft delete: todos los GET filtran `WHERE deleted_at IS NULL`. Los registros eliminados no son visibles.

5.1 Módulo Auth

Método	Endpoint	Body / Params	Response	Auth requerida
POST	<code>/api/v1/auth/register</code>	<code>{ nombre, apellido, email, password }</code>	201 UserResponse	No
POST	<code>/api/v1/auth/login</code>	<code>{ email, password }</code>	200 TokenResponse	No — rate limited 5/15min
POST	<code>/api/v1/auth/refresh</code>	<code>{ refresh_token }</code>	200 TokenResponse	No
POST	<code>/api/v1/auth/logout</code>	<code>{ refresh_token }</code>	204 No Content	Bearer token
GET	<code>/api/v1/auth/me</code>	—	200 UserResponse	Bearer token

5.2 Módulo Categorías

Método	Endpoint	Descripción	Rol requerido	Response
GET	<code>/api/v1/categorias</code>	Listar categorías (paginado, filtro parent_id)	Público	200 List[CategoriaRead]
GET	<code>/api/v1/categorias/{id}</code>	Categoría por ID con subcategorías anidadas	Público	200 CategoriaRead
POST	<code>/api/v1/categorias</code>	Crear categoría	ADMIN	201 CategoriaRead
PUT	<code>/api/v1/categorias/{id}</code>	Actualizar categoría	ADMIN	200 CategoriaRead
DELETE	<code>/api/v1/categorias/{id}</code>	Soft delete (HTTP 409 si tiene productos activos)	ADMIN	204 No Content

5.3 Módulo Productos

Método	Endpoint	Descripción	Rol requerido	Response
GET	/api/v1/productos	Listar (filtro: categoria, disponible, search, page, size)	Público	200 PaginatedProductos
GET	/api/v1/productos/{id}	Detalle con ingredientes, categorías y stock	Público	200 ProductoDetail
POST	/api/v1/productos	Crear producto con ingredientes y categorías	ADMIN	201 ProductoRead
PUT	/api/v1/productos/{id}	Actualizar producto	ADMIN	200 ProductoRead
PATCH	/api/v1/productos/{id}/disponibilidad	Cambiar disponible (true/false)	ADMIN, STOCK	200 ProductoRead
DELETE	/api/v1/productos/{id}	Soft delete producto	ADMIN	204 No Content
GET	/api/v1/productos/{id}/ingredientes	Listar ingredientes del producto	Público	200 List[IngredienteRead]
POST	/api/v1/productos/{id}/ingredientes	Asociar ingrediente a producto	ADMIN	201 ProductoIngredienteRead
DELETE	/api/v1/productos/{id}/ingredientes/{ing_id}	Quitar ingrediente de producto	ADMIN	204 No Content

5.4 Módulo Pedidos

Método	Endpoint	Descripción	Rol requerido	Response
GET	/api/v1/pedidos	Listar pedidos propios (CLIENT) o todos (ADMIN/PEDIDOS)	CLIENT/ADMIN/PEDIDOS	200 PaginatedPedidos
GET	/api/v1/pedidos/{id}	Detalle completo con líneas, trazabilidad y estado de pago	Propietario/ADMIN	200 PedidoDetail
POST	/api/v1/pedidos	Crear pedido desde carrito. Todo en una transacción (UoW).	CLIENT	201 PedidoRead
PATCH	/api/v1/pedidos/{id}/estado	Avanzar estado. Valida máquina de estados. UoW atómico.	ADMIN/PEDIDOS	200 PedidoRead
GET	/api/v1/pedidos/{id}/historial	Historial completo de transiciones. ORDER BY created_at ASC.	Propietario/ADMIN	200 List[HistorialRead]
DELETE	/api/v1/pedidos/{id}	Cancelar pedido propio (solo desde PENDIENTE o CONFIRMADO).	CLIENT propietario	200 PedidoRead

5.5 Módulo Pagos (MercadoPago) ★ NUEVO

Método	Endpoint	Descripción	Rol requerido	Response
POST	/api/v1/pagos/crear	Crea pago con token de tarjeta. Registra en tabla Pago. Retorna estado MP.	CLIENT	201 PagoResponse
POST	/api/v1/pagos/webhook	Endpoint IPN de MercadoPago. Actualiza estado del pago y del pedido.	Público (validar firma)	200 { status: ok }
GET	/api/v1/pagos/{pedido_id}	Consulta el pago asociado a un pedido.	Propietario / ADMIN	200 PagoResponse

5.6 Módulo Direcciones ★ NUEVO

Método	Endpoint	Descripción	Rol requerido	Response
GET	/api/v1/direcciones	Listar direcciones del usuario autenticado	CLIENT	200 List[DireccionRead]
POST	/api/v1/direcciones	Crear nueva dirección de entrega	CLIENT	201 DireccionRead
PUT	/api/v1/direcciones/{id}	Actualizar dirección propia	CLIENT	200 DireccionRead
DELETE	/api/v1/direcciones/{id}	Soft delete dirección propia	CLIENT	204 No Content
PATCH	/api/v1/direcciones/{id}/principal	Marcar como dirección principal	CLIENT	200 DireccionRead

5.7 Módulo Admin

Método	Endpoint	Descripción	Rol requerido	Response
GET	/api/v1/admin/dashboard	KPIs: ventas hoy/semana/mes, pedidos por estado, top 5 productos	ADMIN	200 DashboardStats
GET	/api/v1/admin/usuarios	Listar usuarios paginado con filtro por rol	ADMIN	200 PaginatedUsuarios
PUT	/api/v1/admin/usuarios/{id}	Actualizar datos de usuario	ADMIN	200 UsuarioRead
DELETE	/api/v1/admin/usuarios/{id}	Soft delete de usuario	ADMIN	204 No Content

Método	Endpoint	Descripción	Rol requerido	Response
POST	/api/v1/admin/usuarios/{id}/roles	Asignar rol a usuario	ADMIN	200 UsuarioRead
GET	/api/v1/admin/stock	Listar productos con stock_cantidad y disponible	ADMIN, STOCK	200 List[StockView]
PATCH	/api/v1/admin/stock/{id}	Actualizar stock_cantidad de un producto	ADMIN, STOCK	200 StockView

6. Schemas de Request / Response (Pydantic v2)

Todos los schemas usan Pydantic v2. Se definen schemas separados para Create, Update y Read. Nunca se expone el model de SQLAlchemy directamente como response.

6.1 Auth

Schema	Campos requeridos	Validaciones
LoginRequest	email: EmailStr, password: str	password mínimo 8 caracteres
RegisterRequest	nombre: str, apellido: str, email: EmailStr, password: str	nombre/apellido min 2 max 80. password min 8. Unicidad de email verificada en servicio.
TokenResponse	access_token: str, refresh_token: str, token_type: str, expires_in: int	token_type = 'bearer'. expires_in en segundos.
UserResponse	id: int, nombre: str, apellido: str, email: str, roles: list[str], created_at: datetime	Nunca incluye password_hash.

6.2 Pedidos

Schema	Campos	Validaciones / Notas
CrearPedidoRequest	items: list[ItemPedidoRequest], forma_pago_codigo: str, direccion_id: int None, notas: str None	Mínimo 1 item. forma_pago_codigo debe existir en catálogo FormaPago.
ItemPedidoRequest	producto_id: int, cantidad: int, personalizacion: list[int] None	cantidad ≥ 1. personalizacion = IDs de ingredientes removidos (corregido de dict en v2.0).
AvanzarEstadoRequest	nuevo_estado: str, motivo: str None	motivo obligatorio si nuevo_estado = CANCELADO.
PedidoRead	id, estado_codigo, total, created_at	Versión compacta para listados.
PedidoDetail	id, estado_codigo, estado_descripcion, subtotal, descuento, costo_envio, total, items, historial, pago, created_at	Versión completa para vista de detalle.
HistorialEstadoRead	estado_desde: str None, estado_hacia: str, motivo: str None, usuario_nombre: str None, created_at: datetime	estado_desde = None en el primer registro.
DetallePedidoRead	producto_id, nombre_snapshot, precio_snapshot, cantidad, personalizacion	Snapshot: precio y nombre no reflejan cambios posteriores al producto.

6.3 Pagos

Schema	Campos	Notas
PagoCreateRequest	pedido_id: int, card_token: str, payment_method_id: str, installments: int = 1	card_token generado por MercadoPago.js en el frontend (PCI compliant).
PagoResponse	id: int None, status: str, status_detail: str None, transaction_amount: float, payment_method_id: str None	id = mp_payment_id. status: pending/approved/rejected.

6.4 Ingrediente (corregido) ★

Schema	Campos	Notas
IngredienteRead	id: int, nombre: str, descripcion: str None, es_alergeno: bool	Incluye es_alergeno para badge en UI.
IngredienteCreate	nombre: str, descripcion: str None, es_alergeno: bool = False	
ProductoIngredienteRead	producto_id, ingrediente: IngredienteRead, es_removable: bool	Relación con metadato es_removable.

7. Patrón Unit of Work (UoW) — Especificación

El Unit of Work actúa como director de orquesta que garantiza que todas las operaciones de base de datos dentro de una transacción de negocio tengan éxito o fallen como un conjunto. Su implementación resuelve la ambigüedad arquitectónica detectada en la auditoría (sección 4.3): el commit ya no ocurre en el service.

7.1 Responsabilidades y contratos

Aspecto	Especificación
Ubicación	app/core/uow.py — infraestructura transversal, importado por services, nunca por routers
Ciclo de vida	El UoW abre la sesión en <code>__init__</code> , hace <code>commit()</code> automático en <code>__exit__</code> sin excepciones, <code>rollback()</code> si hay excepción, cierra la sesión siempre.
Uso en router	El router abre el contexto: <code>with UnitOfWork()</code> as <code>uow</code> : — pasa <code>uow</code> al método del service.
Uso en service	El service llama <code>uow.repos.metodo()</code> . Nunca llama <code>uow.session.commit()</code> directamente.
Repositorios	El UoW expone todos los repositorios como atributos: <code>uow.pedidos</code> , <code>uow.productos</code> , <code>uow.historial</code> , <code>uow.pagos</code> , <code>uow.direcciones</code> , <code>uow.usuarios</code> .
Flush intermedio	El service puede llamar <code>uow.flush()</code> para obtener IDs sin hacer commit. Ejemplo: crear un Pedido y luego crear sus DetallePedido con el id generado.
Atomicidad	Si <code>crear_pedido()</code> falla al insertar un DetallePedido, el rollback deshace el Pedido y el HistorialEstadoPedido creados en la misma transacción.

7.2 BaseRepository[T] genérico

Todos los repositorios heredan de `BaseRepository[T]`. Los métodos comunes son provistos por la clase base. Cada repositorio específico agrega solo las queries propias de su dominio.

Método en <code>BaseRepository[T]</code>	Descripción
<code>get_by_id(entity_id: int) → T None</code>	Obtiene entidad por clave primaria. Retorna None si no existe.
<code>list_all(skip: int, limit: int) → list[T]</code>	Listado simple sin filtros. Para listados complejos, el repo específico define su propio método.
<code>count() → int</code>	Cantidad total de registros. Útil para paginación.
<code>create(entity: T) → T</code>	Agrega a sesión + <code>flush()</code> + <code>refresh()</code> . Retorna entidad con ID asignado.
<code>update(entity: T) → T</code>	Agrega entidad modificada a sesión + <code>flush()</code> + <code>refresh()</code> .
<code>delete(entity: T) → None</code>	Hard delete. Solo se usa cuando el modelo no tiene soft-delete.

7.3 Flujo de una operación con UoW — Crear Pedido

Pas o	Capa	Operación	¿Toca BD?
1	Router	Recibe POST /api/v1/pedidos. Valida body con CrearPedidoRequest.	No
2	Router	Abre contexto: with UnitOfWork() as uow: — llama service.crear_pedido(uow, body, usuario_id).	No
3	Service	Itera items. Para cada uno: uow.productos.get_by_id(). Verifica disponible = true.	Lectura
4	Service	Calcula total como suma de precio_snapshot × cantidad.	No
5	Service	Llama uow.pedidos.create(pedido). uow.flush() → obtiene pedido.id.	INSERT + flush
6	Service	Crea DetallePedido por cada item con nombre_snapshot y precio_snapshot.	INSERT × N
7	Service	Crea primer HistorialEstadoPedido con estado_desde=None (RN-02).	INSERT
8	UoW	__exit__ sin excepción → session.commit(). Todo persiste atómicamente.	COMMIT
9	Router	Serializa pedido con PedidoRead.model_validate(pedido). Retorna HTTP 201.	No
ERR	UoW	Si cualquier paso 3-7 lanza excepción → __exit__ llama rollback(). Nada persiste.	ROLLBACK

8. Integración MercadoPago — Especificación Técnica

Food Store integra MercadoPago Checkout API mediante el modelo Orders (API recomendada 2025). Permite procesar pagos con tarjeta de crédito/débito, Rapipago, Pago Fácil y Cuenta MercadoPago sin redirigir al cliente fuera del sitio.

¿Por qué Checkout API con Orders?

Única integración para múltiples medios de pago. Sin múltiples APIs separadas.

Procesamiento en modo automático o manual según necesidad.

Datos de tarjeta tokenizados por MercadoPago.js — nunca pasan por el servidor de Food Store (PCI SAQ-A compliant).

Notificaciones push (IPN/webhook) para confirmación asíncrona del pago.

8.1 Flujo completo de pago

Pas o	Actor	Acción	Resultado
1	Frontend	Renderiza el componente CardPayment del SDK @mercadopago/sdk-react (iframe PCI-compliant).	Formulario seguro de tarjeta
2	Frontend	El cliente ingresa datos de tarjeta. SDK tokeniza → genera card_token.	card_token (nunca el número real)
3	Frontend	Llama al hook useCrearPago con { pedido_id, card_token, payment_method_id, installments }.	POST /api/v1/pagos/crear
4	Backend	Service genera idempotency_key (UUID). Llama SDK Python: sdk.payment().create() con idempotency_key.	Respuesta MP con id + status
5	Backend	UoW persiste registro en tabla Pago con mp_payment_id, mp_status y external_reference.	INSERT en tabla pagos
6	MP → Backend	MercadoPago envía POST a /api/v1/pagos/webhook con topic=payment y el ID del pago.	Notificación IPN
7	Backend	Webhook consulta el pago con SDK: sdk.payment().get(id). Si status = approved → avanza pedido a CONFIRMADO vía UoW.	PATCH estado en tabla pedidos + historial
8	Frontend	Polling cada 10s con useQuery refetchInterval. Detecta cambio a CONFIRMADO.	UI actualizada automáticamente

8.2 Configuración requerida

Variable de entorno	Descripción	Ambiente
MP_ACCESS_TOKEN	Access Token de la aplicación MP. Clave privada — solo en backend.	TEST-xxx / APP_USR-xxx
MP_PUBLIC_KEY	Public Key de la aplicación MP. Se expone al frontend vía VITE_MP_PUBLIC_KEY.	TEST-xxx / APP_USR-xxx
MP_NOTIFICATION_URL	URL pública donde MP enviará el webhook IPN.	https://dominio.com/api/v1/pagos/webhook
VITE_MP_PUBLIC_KEY	Public Key para el SDK de React. Prefijo VITE_ para que Vite la exponga.	Igual que MP_PUBLIC_KEY

Credenciales de test — cómo obtenerlas

1. Crear cuenta en <https://www.mercadopago.com.ar/developers/>
2. Crear una aplicación en el panel de desarrolladores.
3. En 'Credenciales de prueba': copiar TEST Access Token y TEST Public Key.
4. Usar tarjetas de prueba del sandbox para testear sin procesar cobros reales.

8.3 Especificación del módulo pagos — Backend

8.3.1 Responsabilidades del PagoService

Método	Parámetros	Responsabilidad	Retorna
crear_pago()	uow, request, pedido_id, payer_email	Genera idempotency_key UUID. Construye payload MP. Llama sdk.payment().create(). Persiste Pago en BD vía uow. Si status = rejected → HTTPException 402.	dict con respuesta MP
procesar_webhook()	uow, topic, resource_id, pedido_service	Solo procesa topic = 'payment'. Consulta sdk.payment().get(id). Actualiza mp_status en BD. Si approved → avanza pedido a CONFIRMADO.	None

8.3.2 Contrato del endpoint de pago

Aspecto	Especificación
Idempotencia	El backend genera un UUID como idempotency_key y lo envía en el header x-idempotency-key del SDK MP. Evita cobros duplicados por reintento.
Atomicidad	Crear el Pago en BD y actualizar el estado del Pedido se hacen en el mismo contexto UoW.
Rechazo de pago	Si mp_status = rejected, se retorna HTTP 402 Payment Required con el campo status_detail para mostrar al usuario.

Aspecto	Especificación
Pago pendiente	Si <code>mp_status = pending</code> (pago en efectivo), el pedido permanece en PENDIENTE hasta que el webhook confirme el pago.
Seguridad webhook	Validar que el request viene de MercadoPago verificando la firma X-Signature con el <code>MP_WEBHOOK_SECRET</code> .

8.4 Especificación del componente de pago — Frontend

Aspecto	Especificación
Inicialización SDK	<code>initMercadoPago(VITE_MP_PUBLIC_KEY, { locale: 'es-AR' })</code> debe llamarse una sola vez, en el componente raíz o en <code>App.tsx</code> .
Componente	Usar <code><CardPayment></code> del SDK de React. Provee el formulario tokenizado en <code>iframe</code> . Los datos de tarjeta nunca llegan al código de Food Store.
<code>onSubmit</code>	El callback recibe <code>{ token, payment_method_id, installments }</code> . El componente llama al hook <code>useCrearPago</code> con esos datos más el <code>pedido_id</code> .
Estado de pago	El <code>paymentStore</code> gestiona status: <code>'idle' 'processing' 'approved' 'rejected' 'error'</code> . La UI reacciona a este estado.
Feedback al usuario	Mostrar mensaje de carga durante el procesamiento. Si <code>rejected</code> : mostrar <code>status_detail</code> traducido. Si <code>approved</code> : limpiar carrito y redirigir.
Medios alternativos	Para RapiPago/Pago Fácil se puede usar el componente <code><PaymentForm></code> del SDK o redirigir al Checkout Pro. Fuera de alcance v1.

8.5 Estados de pago MercadoPago y acciones del sistema

Estado MP	Descripción	Acción en Food Store
<code>pending</code>	Pago iniciado o en proceso (efectivo pendiente de cobro)	Pedido permanece en PENDIENTE. El webhook confirmará cuando se acredite.
<code>approved</code>	Pago aprobado y acreditado	Webhook avanza el pedido a CONFIRMADO de forma automática vía UoW.
<code>rejected</code>	Pago rechazado por el banco o MP	Se muestra <code>status_detail</code> al cliente. El pedido permanece en PENDIENTE.
<code>in_process</code>	Pago en revisión manual por MP	Pedido permanece en PENDIENTE. El webhook notificará la resolución.
<code>cancelled</code>	Pago cancelado	El cliente puede reintentar o cancelar el pedido.

8.6 Tarjetas de prueba — Sandbox

Número de tarjeta	Red	Resultado	CVV	Vencimiento
4509 9535 6623 3704	Visa	Pago aprobado	123	11/25

Número de tarjeta	Red	Resultado	CVV	Vencimiento
3714 496353 98431	American Express	Pago aprobado	1234	11/25
4000 0000 0000 0002	Visa	Pago rechazado	123	11/25

9. Gestión de Estado con Zustand — Especificación Completa

Zustand es la librería de gestión de estado global del frontend. Se eligió sobre Redux por su API mínima sin boilerplate, excelente integración con React hooks y soporte nativo de middleware persist. Food Store requiere cuatro stores con responsabilidades claramente separadas.

9.1 Cuatro stores requeridos

Store	Archivo	Estado que gestiona	Middleware	Persiste entre recargas
authStore	store/authStore.ts	accessToken, usuario, isAuthenticated	persist	Sí — solo el accessToken
cartStore	store/cartStore.ts	items del carrito, cantidades, personalizaciones	persist	Sí — items completos
paymentStore	store/paymentStore.ts	Estado del proceso de pago MP: status, mpPaymentId	Ninguno (sesión)	No — se resetea al recargar
uiStore	store/uiStore.ts	cartOpen, sidebarOpen, confirmModal activo	Ninguno	No

9.2 authStore — Contrato

Elemento	Tipo	Descripción
accessToken	string null	JWT de acceso. Se actualiza en login y refreshToken.
usuario	UserResponse null	Datos del usuario autenticado.
isAuthenticated	boolean	true cuando accessToken != null y no expirado.
login(credentials)	(LoginRequest) → Promise<void>	Llama authApi.login(). Guarda token y usuario en el store.
logout()	() → Promise<void>	Llama authApi.logout(). Limpia token y usuario. Redirige al login.
refreshToken()	() → Promise<void>	Llamado por el interceptor Axios en HTTP 401. Renueva accessToken sin requerir re-login.
hasRole(role)	(string) → boolean	Verifica si usuario.roles incluye el rol solicitado. Útil para ProtectedRoute.

persist — authStore

Solo se persiste el accessToken (no el objeto usuario completo).

Al recargar, si hay accessToken en localStorage, el interceptor lo usará automáticamente.

El objeto usuario se puede reconstruir llamando GET /api/v1/auth/me al inicializar la app.
Usar partialize: (state) => ({ accessToken: state.accessToken }) en el middleware persist.

9.3 cartStore — Contrato

Elemento	Tipo	Descripción
items	CartItem[]	Lista de items en el carrito. CartItem = { producto_id, nombre, precio, cantidad, imagen_url?, personalizacion?: number[] }
addItem(item)	(CartItem) → void	Si el producto ya existe → incrementa cantidad. Si no → agrega nuevo item.
removeItem(id)	(number) → void	Elimina el item del carrito por producto_id.
updateCantidad(id, cant)	(number, number) → void	Actualiza cantidad. Mínimo 1.
clearCart()	() → void	Vacía el carrito. Se llama tras checkout exitoso.
itemCount()	() → number	Total de unidades en el carrito. Para el badge en el ícono del carrito.
subtotal()	() → number	Suma de precio × cantidad de todos los items.
costoEnvio()	() → number	Retorna 50.00 si hay items, 0 si el carrito está vacío.
total()	() → number	subtotal() + costoEnvio().

persist — cartStore

Se persiste el array items completo con la clave 'foodstore-cart' en localStorage.
version: 1 — incrementar si cambia la estructura de CartItem (Zustand migra automáticamente).
Al completar el checkout exitosamente, llamar clearCart() para vaciar el carrito persistido.

9.4 paymentStore — Contrato

Elemento	Tipo	Descripción
status	'idle' 'processing' 'approved' 'rejected' 'error'	Estado del proceso de pago en curso. Controla qué renderiza la UI de checkout.
mpPaymentId	number null	ID del pago en MercadoPago. Llega en la respuesta de crear_pago.
statusDetail	string null	Detalle de rechazo de MP. Ej: 'cc_rejected_insufficient_amount'. Se muestra al usuario traducido.

Elemento	Tipo	Descripción
setPaymentStatus(status, detail?, id?)	(...) → void	Actualiza el estado del pago. Llamado por el hook useCrearPago en onSuccess/onError.
reset()	() → void	Resetea al estado idle. Llamado al iniciar un nuevo intento de pago.

9.5 uiStore — Contrato

Elemento	Tipo	Descripción
cartOpen	boolean	true cuando el drawer del carrito está abierto.
sidebarOpen	boolean	true cuando el sidebar de admin está abierto (en mobile).
confirmModal	ConfirmModal null	Datos del modal de confirmación activo: { open, title, message, onConfirm }. null = cerrado.
openCart()	() → void	Abre el CartDrawer.
closeCart()	() → void	Cierra el CartDrawer.
toggleSidebar()	() → void	Alterna el estado del sidebar de admin.
openConfirmModal(title, onConfirm)	(...) → void	Abre el modal de confirmación para acciones destructivas.
closeConfirmModal()	() → void	Cierra y limpia el confirmModal.

9.6 Buenas prácticas de consumo de stores

Práctica	Descripción	Ejemplo
Suscripción por slice	Seleccionar solo el estado necesario. Evita re-renders innecesarios cuando cambian otros campos del store.	<code>const itemCount = useCartStore(s => s.itemCount())</code>
Acceso a actions	Las actions no cambian → extraerlas por destructuring no causa re-renders.	<code>const { addItem } = useCartStore()</code>
Evitar el store completo	Nunca suscribirse al store completo sin selector. Causa re-render ante cualquier cambio.	✗ <code>const store = useCartStore()</code>
Estado de pago	El paymentStore debe resetearse al iniciar cada intento de pago, no solo al completar.	<code>reset()</code> en el <code>onClick</code> del botón pagar
Zustand fuera de React	Para acceder al estado en interceptors Axios (no en componente): <code>useStore.getState().campo</code>	<code>useAuthStore.getState().accessToken</code>



10. Funcionalidades Frontend por Módulo

10.1 Módulo Auth

Login — /pages/auth/login/

Aspecto	Especificación
Formulario	TanStack Form. Campos: email (type=email) y password (type=password).
Validaciones	email válido. password mínimo 8 caracteres. Errores inline bajo cada campo.
Submit	Botón deshabilitado y con spinner durante la petición. Usa useMutation de TanStack Query.
Redirección	Post-login según rol: ADMIN/STOCK/PEDIDOS → /admin/dashboard, CLIENT → /store.
Errores	Mensaje genérico si credenciales inválidas. No revelar cuál campo falló (seguridad).
Backend	POST /api/v1/auth/login. Rate limiting: HTTP 429 si supera 5 intentos en 15 min.

Registro — /pages/auth/register/

Aspecto	Especificación
Formulario	Campos: nombre, apellido, email, password, confirmarPassword.
Validaciones	confirmarPassword === password. Email único verificado on-blur con debounce. password ≥ 8.
Auto-login	Tras registro exitoso, solicitar token automáticamente y redirigir al store.
Backend	POST /api/v1/auth/register. HTTP 409 si email ya existe. Asigna rol CLIENT automáticamente.

10.2 Módulo Cliente — Store

Catálogo — /pages/store/home/

Aspecto	Especificación
Grid	ProductCard con imagen, nombre, precio, badge de alérgenos y botón 'Agregar al carrito'.
Búsqueda	Input con debounce 300ms. Filtra por nombre vía TanStack Query (queryKey incluye el término).
Filtro	Sidebar o tabs con categorías del API. Click actualiza el queryKey.
Paginación	Paginación numérica usando GET ?page=&size=20. Mostrar total de páginas.
Sin stock	Badge 'Sin stock' sobre la card. Botón 'Agregar' deshabilitado si disponible = false.
Loading	Skeleton loaders al cargar. Nunca spinner global.

Aspecto	Especificación
Empty state	Ilustración + texto 'No encontramos productos' si la búsqueda no retorna resultados.

Detalle de Producto — /pages/store/productDetail/

Aspecto	Especificación
Galería	Imágenes del producto. Primera imagen como principal.
Ingredientes	Lista de ingredientes con badge de alérgeno si es_alergeno = true.
Personalización	Checkboxes de ingredientes es_removible = true. Seleccionados = removidos del pedido.
Cantidad	Input numérico mínimo 1. Actualiza el subtotal mostrado en tiempo real.
Carrito	Botón 'Agregar al carrito' llama cartStore.addItem() con la personalización seleccionada.

Carrito — CartDrawer o /pages/store/cart/

Aspecto	Especificación
Items	Lista desde cartStore. Cada ítem con imagen, nombre, cantidad editable y botón eliminar.
Totales	Subtotal, costo de envío (\$50 fijo) y total calculados en tiempo real por cartStore.
Vaciar	Botón 'Vaciar carrito' abre modal de confirmación (uiStore.openConfirmModal).
Persistencia	Zustand persist: al recargar la página el carrito se restaura desde localStorage.
Acción	Botón 'Ir al checkout' → navega a /store/checkout si hay ítems. Deshabilitado si vacío.

Checkout — /pages/store/checkout/

Aspecto	Especificación
Resumen	Items del cartStore con subtotal, costo de envío y total.
Dirección	Select de direcciones del usuario (GET /api/v1/direcciones). Opción para crear nueva dirección inline.
Forma de pago	Select de formas de pago del catálogo. Si = MERCADOPAGO → mostrar componente <CardPayment>.
CardPayment	Componente del SDK @mercadopago/sdk-react. El cliente ingresa datos de tarjeta en iframe seguro.
Flujo de pago	1. POST /api/v1/pedidos (crea el pedido). 2. POST /api/v1/pagos/crear (procesa el pago). 3. Esperar respuesta.
On success	Si pago aprobado: clearCart(), navegar a /client/orders/{pedido_id}.
On rejected	Mostrar statusDetail traducido. Permitir reintentar sin salir del checkout.

Aspecto	Especificación
Validación	Mínimo 1 ítem, dirección seleccionada (o null si retiro), forma de pago seleccionada.

10.3 Módulo Cliente — Mis Pedidos

Aspecto	Especificación
Lista	GET /api/v1/pedidos paginado. Badge de estado coloreado (OrderStatusBadge).
Polling	refreshInterval: 30_000 en useQuery. Actualiza estado del pedido en tiempo real.
Detalle	Click en pedido → /client/orders/{id}. Detalle con ítems, totales, estado de pago e historial.
Timeline	HistorialTimeline: cada transición de estado con timestamp, actor y motivo (si lo hay).
Cancelar	Botón 'Cancelar' visible solo si estado es PENDIENTE o CONFIRMADO. Abre modal de confirmación.
Estado de pago	Muestra si el pago fue aprobado, rechazado o pendiente con el ícono/badge correspondiente.

10.4 Panel de Administración

Dashboard — /pages/admin/adminHome/

Aspecto	Especificación
KPIs	Cards: pedidos hoy, ingresos hoy, productos activos, usuarios activos.
Gráfico	recharts — gráfico de barra o pie con pedidos por estado.
Tabla	Últimos 10 pedidos con link al detalle.
Top	Top 5 productos más vendidos por volumen de unidades.
Backend	GET /api/v1/admin/dashboard — queries agregadas en el servicio. Nunca en el router.

CRUD Categorías — /pages/admin/categories/

Aspecto	Especificación
Tabla	Nombre, categoría padre, cantidad de productos asociados, acciones.
Modal	Crear/editar: nombre, descripción, categoría padre (select con árbol anidado).
Eliminar	Confirmación modal. HTTP 409 si tiene productos activos → mostrar mensaje de advertencia.

CRUD Productos — /pages/admin/products/

Aspecto	Especificación
Tabla	Imagen, nombre, precio, stock_cantidad, disponible (toggle), acciones.
Modal	Crear/editar: nombre, descripción, precio, URL imagen, categorías (multi-select), ingredientes (multi-select con es_removable).
Toggle	Disponible se cambia desde la tabla con PATCH optimista. Feedback inmediato, rollback si falla.
Validaciones	precio ≥ 0 , al menos una categoría, nombre no duplicado.

Gestión de Pedidos Admin — /pages/admin/orders/

Aspecto	Especificación
Tabla	Todos los pedidos. Filtros: estado, fecha, búsqueda por ID o email de cliente.
Avanzar	Selector de estado que muestra solo las transiciones válidas para el estado actual.
Modal motivo	Al avanzar a CANCELADO: campo motivo obligatorio.
Panel	Panel lateral con historial de trazabilidad del pedido seleccionado.
Estado pago	Columna de estado MP: approved/pending/rejected con badge de color.

Gestión de Stock — /pages/admin/stock/

Aspecto	Especificación
Vista	Tabla con producto, stock_cantidad, disponible, última actualización.
Editar stock	Input numérico inline. PATCH /api/v1/admin/stock/{id}. Mínimo 0.
Toggle	Disponible se puede cambiar independientemente del stock.

11. Diseño y UX

11.1 Sistema de diseño

Elemento	Especificación	Implementación
Paleta primaria	#1F3864 (navy), #2E75B6 (blue), #4472C4 (accent)	Tailwind custom colors en tailwind.config.ts
Paleta semántica	PENDIENTE=amber, CONFIRMADO=blue, EN_PREP=cyan, EN_CAMINO=violet, ENTREGADO=green, CANCELADO=red	Clases condicionales en OrderStatusBadge
Paleta MercadoPago	approved=green, pending=amber, rejected=red	Badge de estado de pago
Tipografía	Inter (sans-serif), JetBrains Mono (código)	Google Fonts vía CDN en index.html
Espaciado	Grid base 4px (Tailwind default). Múltiplos de 4.	p-2=8px, p-4=16px, p-8=32px
Breakpoints	sm=640px, md=768px, lg=1024px, xl=1280px	Tailwind defaults — mobile-first
Iconos	Heroicons (outline) vía @heroicons/react	Tree-shaking: importar solo los usados

11.2 Patrones de UX obligatorios

Patrón	Descripción	Componente / Hook
Skeleton loaders	En listas, cards y tablas mientras carga. Nunca spinner global.	Skeleton.tsx — useProductos, usePedidos
Error states	Mensaje inline + botón 'Reintentar' para errores de red.	Manejo de isError en useQuery
Empty states	Ilustración + texto descriptivo + CTA cuando no hay datos.	En CatalogoGrid, PedidosList
Toast notifications	Feedback de acciones exitosas y errores. Usar Sonner o react-hot-toast.	Llamado en onSuccess/onError de useMutation
Confirmación modal	Para acciones destructivas: eliminar categoría, cancelar pedido, vaciar carrito.	uiStore.openConfirmModal + ConfirmModal.tsx
Optimistic updates	Toggle de disponibilidad y cambios menores: actualización inmediata con rollback.	useMutation con onMutate + onError
Responsive	Layout mobile-first. Sidebar colapsable en mobile. Grid adaptativo.	Tailwind breakpoints en todos los layouts

12. Configuración y Setup

12.1 Variables de entorno — Completas v3.0

Variable	Descripción	Valor ejemplo
DATABASE_URL	Conexión a PostgreSQL	postgresql://user:pass@localhost:5432/foodstore_db
SECRET_KEY	Clave secreta para firmar JWT (mín. 32 chars)	your-super-secret-key-min-32-chars
ALGORITHM	Algoritmo JWT	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	Expiración del access token en minutos	30
REFRESH_TOKEN_EXPIRE_DAYS	Expiración del refresh token en días	7
CORS_ORIGINS	Orígenes permitidos (JSON array)	["http://localhost:5173"]
MP_ACCESS_TOKEN	Access Token de MercadoPago (backend)	TEST-xxxx
MP_PUBLIC_KEY	Public Key de MercadoPago (para el frontend)	TEST-xxxx
MP_NOTIFICATION_URL	URL del webhook IPN de MercadoPago	https://dominio.com/api/v1/pagos/webhook
VITE_API_URL	URL base del backend (Vite — frontend)	http://localhost:8000
VITE_MP_PUBLIC_KEY	Public Key MP expuesta al frontend vía Vite	TEST-xxxx

12.2 Seed data obligatorio — app/db/seed.py

El script seed.py debe ejecutarse una vez después de alembic upgrade head. Sin este paso, la aplicación no funciona: no existen roles, estados de pedido ni formas de pago.

Entidad	Registros a insertar
Rol	ADMIN, STOCK, PEDIDOS, CLIENT — los cuatro roles del sistema RBAC
EstadoPedido	PENDIENTE, CONFIRMADO, EN_PREP, EN_CAMINO, ENTREGADO, CANCELADO — con es_terminal correspondiente
FormaPago	MERCADOPAGO (habilitado), EFECTIVO (habilitado), TRANSFERENCIA (habilitado)
Usuario admin	admin@foodstore.com / Admin1234! — con rol ADMIN asignado. Contraseña debe cambiarse en producción.

12.3 CORS — Configuración en main.py

Parámetro	Valor	Descripción
allow_origins	settings.CORS_ORIGINS	Lista desde .env. Ejemplo: ["http://localhost:5173"]
allow_credentials	True	Necesario para enviar cookies httpOnly del refresh token
allow_methods	["*"]	Todos los métodos HTTP
allow_headers	["*"]	Todos los headers, incluyendo Authorization

13. Rúbrica de Corrección — v3.0

Puntaje total: 200 puntos. Corrección escrita + video de demostración obligatorio.

13.1 Backend — Estructura y Configuración (10 pts)

Criterio	Excelente (10)	Bueno (7-9)	Regular (4-6)	Insuficiente (0-3)
Estructura del proyecto	Capas router/service/uow/repository/model. Módulos separados por dominio. core/ separado.	Separación parcial, algunos módulos mezclados.	Estructura plana o sin capas claras.	Sin estructura reconocible.
Configuración de BD	Alembic + migrations versionadas + seed.py ejecutable. Variables en .env.	Alembic presente, seed parcial.	Solo create_all(), sin Alembic.	BD hardcodeada.
CORS + Rate limiting	CORSMiddleware configurado + slowapi en login.	Solo CORS o solo rate limiting.	CORS hardcodeado o incompleto.	Sin CORS.

13.2 Backend — Modelo de Datos (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
Models y Schemas	SQLModel correcto, constraints, soft-delete, snapshot, entidades completas (Ingrediente, FormaPago, DireccionEntrega, RefreshToken, Pago).	Models correctos, faltan algunas entidades o constraints.	Models básicos sin patrones avanzados.	Models incorrectos o incompletos.
Relaciones	3FN, FK con ON DELETE correcto, tablas pivot PK compuesta, self-ref en Categoria.	Relaciones correctas, falta alguna FK o cascada.	Relaciones básicas sin FK explícitas.	Sin relaciones correctas.
Validaciones	Pydantic v2 con Field validators, EmailStr, ge/le, schemas separados Read/Create/Update.	Validaciones básicas, sin validators custom.	Poco uso de Field validators.	Sin validaciones.

13.3 Backend — Unit of Work y Repository (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
Unit of Work	UoW completo con context manager, commit/rollback automático, provee todos los repositorios. Ningún service hace session.commit() directamente.	UoW presente pero incompleto o el service hace commit manual.	Sin UoW, transacciones manuales.	Sin gestión de transacciones.
BaseRepository	BaseRepository[T] genérico con CRUD base. Todos los repos heredan y extienden con sus queries propias.	BaseRepository presente, algunos repos no heredan.	Sin BaseRepository genérico.	Sin patrón Repository.
Queries	Repos con filtros, paginación, joins, ORDER BY, manejo de soft-delete en queries.	Paginación presente, joins parciales.	Solo CRUD básico sin filtros.	Sin queries personalizadas.

13.4 Backend — Capa de Servicio (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
Lógica de negocio	State machine implementada en servicio. RN-01/02/03 validadas. Servicios stateless.	Lógica presente pero algunas RN en routers.	Lógica básica sin validaciones de negocio.	Sin capa de servicio.
Uso de UoW	El service recibe uow por parámetro y opera sobre uow.repos. Nunca gestiona sesión directamente.	Service usa UoW parcialmente.	Service mezcla UoW y sesión directa.	Sin uso de UoW.
Errores	HTTPException con código semántico (404, 409, 422, 402), mensajes descriptivos.	HTTPException presente, algunos códigos incorrectos.	Excepciones genéricas o 500 en todos.	Sin manejo de errores.

13.5 Backend — Controladores REST (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
Endpoints RESTful	Verbos HTTP correctos, rutas semánticas, status codes precisos	Verbos y rutas correctas, algunos status	Verbos incorrectos o	Sin convenciones REST.

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
	(201/204/422/402), prefijo /api/v1 unificado.	codes incorrectos.	rutas no semánticas.	
Schemas	Schemas Pydantic separados (Read/Create/Update), response_model en todos los endpoints.	Schemas presentes pero reutilizados incorrectamente .	Schemas básicos o return directo del model.	Sin schemas.
CRUD completo	Create/Read/Update/Delete (soft) para todas las entidades. Pedidos con state machine.	CRUD presente en mayoría de entidades, falta alguno.	CRUD parcial, falta Update o Delete.	Solo lectura.

13.6 Backend — MercadoPago (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
Integración SDK	SDK Python configurado con access token. Pago creado correctamente con idempotency_key UUID.	SDK configurado, sin idempotency_key.	Integración parcial o hardcodeada.	Sin integración MP.
Webhook IPN	Endpoint /pagos/webhook que procesa topic=payment, consulta el estado con SDK y avanza el pedido a CONFIRMADO vía UoW si aprobado.	Webhook presente, lógica incompleta.	Webhook definido pero sin lógica.	Sin webhook.
Tabla Pago	Registro de transacciones en BD con mp_payment_id, status, status_detail, external_reference, idempotency_key.	Tabla presente, faltan campos.	Tabla mínima.	Sin registro en BD.

13.7 Frontend — Estructura y TypeScript (10 pts)

Criterio	Excelente (10)	Bueno (7-9)	Regular (4-6)	Insuficiente (0-3)
Estructura	Feature-sliced: pages/features/components/hooks/store/api/types. Sin cross-imports incorrectos.	Estructura presente con algunos módulos mezclados.	Estructura plana.	Sin estructura.

Criterio	Excelente (10)	Bueno (7-9)	Regular (4-6)	Insuficiente (0-3)
TypeScript	Tipos estrictos, no any, componentes tipados, hooks tipados, interfaces en types/.	Mayoría tipado, algunos any o props sin tipar.	Uso básico, muchos any.	Sin TypeScript o sin tipado.
Separación	Lógica de negocio en hooks/stores, UI en components, fetch en api/, tipos en types/.	Separación parcial, algo de lógica en componentes.	Lógica mezclada en componentes.	Sin separación de concerns.

13.8 Frontend — Zustand (10 pts)

Criterio	Excelente (10)	Bueno (7-9)	Regular (4-6)	Insuficiente (0-3)
4 stores requeridos	authStore, cartStore, paymentStore, uiStore correctamente implementados y tipados.	3 stores correctos, falta uno.	Solo authStore y cartStore básicos.	Sin Zustand o un solo store global.
persist correcto	authStore persiste solo el token. cartStore persiste los items completos con versión. paymentStore no persiste.	persist presente pero configuración incorrecta (persiste todo o nada).	persist en algún store sin separación.	Sin persist.
Slices y acceso	Suscripción por slice: useStore(s => s.campo). Actions extraídas sin re-render. getState() en interceptores.	Mayoría de usos correctos, algunos acceden al store completo.	Acceso al store completo en todos lados.	Sin buenas prácticas de consumo.

13.9 Frontend — Integración API y TanStack Query (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
TanStack Query	useQuery/useMutation para todo el fetch. queryKey descriptivos. staleTime configurado. invalidación tras mutaciones.	TanStack Query presente, invalidación parcial.	Fetch directo con useEffect o fetch().	Sin TanStack Query.
Interceptors	Interceptor request inyecta Bearer token. Interceptor response maneja 401 con refresh automático.	Interceptores básicos, sin refresh automático.	Token manual en cada request.	Sin autenticación en requests.

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
MercadoPago frontend	SDK inicializado en App.tsx. <CardPayment> renderizado en checkout. paymentStore actualizado en onSuccess/onError.	SDK integrado, paymentStore parcial.	Formulario propio sin SDK de MP.	Sin integración MP en frontend.

13.10 Frontend — Funcionalidades del Cliente (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
Catálogo	Grid responsive, búsqueda con debounce, filtro por categoría, paginación, skeleton loaders, badge sin-stock.	Catálogo funcional, búsqueda sin debounce o sin paginación.	Lista sin filtros ni paginación.	Sin catálogo funcional.
Carrito	Zustand persist, add/remove/update cantidad, cálculo de total, vaciar con confirmación.	Zustand presente, falta persist o confirmación.	Estado local sin persistencia.	Sin carrito.
Checkout + Pago	Selección de dirección y forma de pago. <CardPayment> de MP. Flujo crear pedido → crear pago → navegar. Limpiar carrito on-success.	Checkout funcional sin MercadoPago integrado.	Checkout básico sin selección de dirección.	Sin checkout.
Historial	Lista paginada, timeline de trazabilidad, polling, estado de pago, cancelación desde estados válidos.	Lista presente, sin trazabilidad o polling.	Lista básica sin detalles.	Sin historial.

13.11 Frontend — Panel de Administración (15 pts)

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
Dashboard	KPIs + gráfico recharts + últimos pedidos + top productos desde endpoint real.	Dashboard con datos reales, gráfico básico.	Dashboard con datos hardcodeados.	Sin dashboard.
CRUD Categorías	Tabla + modal crear/editar + soft delete con validación de	CRUD funcional, falta validación en delete.	Solo lectura o create básico.	Sin CRUD categorías.

Criterio	Excelente (15)	Bueno (10-14)	Regular (5-9)	Insuficiente (0-4)
	productos asociados (HTTP 409).			
CRUD Productos	Tabla con filtros + modal con ingredientes/categorías multi-select + toggle optimista.	CRUD funcional, sin relaciones.	CRUD básico sin relaciones.	Sin CRUD productos.
Gestión Pedidos	Selector de transiciones válidas + modal motivo + panel historial + estado de pago.	Tabla y cambio de estado básico.	Solo visualización.	Sin gestión de pedidos admin.
Gestión Stock	Vista stock_cantidad + toggle disponible + alertas de stock bajo.	Gestión básica sin alertas.	Solo lectura.	Sin gestión de stock.

13.12 UI/UX y Diseño (10 pts)

Criterio	Excelente (10)	Bueno (7-9)	Regular (4-6)	Insuficiente (0-3)
Interfaz	Sistema de diseño consistente: paleta, tipografía, espaciado, componentes reutilizables.	Diseño consistente con pequeñas inconsistencias.	Diseño básico sin sistema.	Sin diseño coherente.
Responsive	Mobile-first, sidebar colapsable, grid adaptativo, probado en 3 breakpoints.	Responsive parcial, algunos elementos rotos en mobile.	Diseño fijo sin responsividad.	Sin responsive.
Feedback	Toasts, skeleton loaders, empty states con CTA, modales de confirmación, estado de pago con badge.	Toasts y loading básicos.	Solo alerts nativos del browser.	Sin feedback visual.

13.13 Calidad de Código y Buenas Prácticas (10 pts)

Criterio	Excelente (10)	Bueno (7-9)	Regular (4-6)	Insuficiente (0-3)
Nomenclatura	snake_case en Python, camelCase en TS, PascalCase en componentes/clases. Nombres descriptivos.	Nomenclatura mayoritariamente correcta.	Mezcla de convenciones.	Sin convenciones.

Criterio	Excelente (10)	Bueno (7-9)	Regular (4-6)	Insuficiente (0-3)
Modularización	Funciones < 50 líneas, SRP, sin código duplicado, hooks personalizados.	Modularización parcial, algunas funciones largas.	Funciones largas, código duplicado.	Sin modularización.
Documentación	Docstrings en clases/funciones públicas Python. JSDoc en hooks y utils. README.md completo con setup.	Comentarios en partes críticas, README básico.	Pocos comentarios, sin README.	Sin documentación.

13.14 Escala de calificación

Rango	Porcentaje	Calificación	Descripción
181-200 pts	90-100%	Excelente	Proyecto completo, profesional, con todas las capas y buenas prácticas.
141-180 pts	70-89%	Bueno	Proyecto funcional con pequeños ajustes o funcionalidades faltantes.
101-140 pts	50-69%	Regular	Proyecto básico con errores o funcionalidades incompletas.
0-100 pts	0-49%	Insuficiente	Proyecto incompleto, no funcional o no sigue la especificación.

Penalizaciones y bonus

- ⚠ Penalización: El proyecto que no corra localmente siguiendo el README tendrá una reducción del 30% del puntaje final.
- ✅ Bonus +10 pts: Tests unitarios con pytest, cobertura > 60% (incluir test_pedidos, test_pagos, test_auth).
- ✅ Bonus +10 pts: Deploy funcional en Railway, Render o Fly.io con URL accesible.
- ⚠ Restricción: El uso de IA generativa sin comprensión demostrable en el video resultará en evaluación oral adicional.

14. Entrega del Proyecto

14.1 Contenido obligatorio

Repositorio GitHub (público)

- Código fuente completo (frontend + backend) en monorepo o dos repos separados
- README.md en cada repo: descripción, requisitos, pasos de instalación y ejecución
- .env.example con todas las variables requeridas incluyendo las de MercadoPago (sin valores reales)
- requirements.txt con versiones fijas para el backend
- package.json con scripts: dev, build, preview, lint
- Script seed.py documentado en el README con instrucciones de ejecución

Documentación técnica

- Screenshots de al menos 10 pantallas: Login, Catálogo, Carrito, Checkout (con CardPayment), Estado de pago, Mis Pedidos, Timeline, Admin Dashboard, Admin Pedidos, Stock
- Diagrama ERD actualizado con todas las entidades de v3.0
- Decisiones técnicas documentadas: UoW vs session manual, Zustand vs Redux, MP Checkout API vs Checkout Pro, stock como campo vs tabla separada

Video demostración (5-10 minutos)

- Flujo cliente: registro → login → catálogo → carrito → checkout → pago con tarjeta de prueba MP → estado aprobado → mis pedidos → trazabilidad
- Flujo admin: dashboard → gestión productos → avance de estado de pedido → historial de trazabilidad → gestión de stock
- Link al video incluido en el README del repositorio principal

14.2 Checklist de entrega

Ítem	Descripción	Estado
CE-01	Link a repositorio GitHub público en la entrega	☐ Pendiente
CE-02	README.md con instrucciones de setup funcionando en máquina limpia	☐ Pendiente
CE-03	.env.example completo con variables de MercadoPago documentadas	☐ Pendiente
CE-04	alembic upgrade head sin errores	☐ Pendiente

Ítem	Descripción	Estado
CE-05	python -m app.db.seed ejecuta correctamente y carga datos iniciales	☐ Pendiente
CE-06	npm install + npm run dev sin errores	☐ Pendiente
CE-07	pip install -r requirements.txt + uvicorn app.main:app sin errores	☐ Pendiente
CE-08	Swagger UI (/docs) accesible con todos los endpoints documentados	☐ Pendiente
CE-09	Pago de prueba con tarjeta sandbox MP funciona end-to-end	☐ Pendiente
CE-10	Unit of Work correctamente implementado (ningún service.session.commit() directo)	☐ Pendiente
CE-11	4 Zustand stores implementados, tipados y con persist correcto	☐ Pendiente
CE-12	Screenshots de al menos 10 pantallas distintas	☐ Pendiente
CE-13	Link a video demostración en README	☐ Pendiente
CE-14	Repositorio público verificado con sesión cerrada	☐ Pendiente

Apéndice — Referencias y Recursos

A. Documentación oficial

Tecnología	URL
FastAPI	https://fastapi.tiangolo.com
SQLModel	https://sqlmodel.tiangolo.com
Pydantic v2	https://docs.pydantic.dev
Alembic	https://alembic.sqlalchemy.org
TanStack Query v5	https://tanstack.com/query
TanStack Form	https://tanstack.com/form
Zustand	https://zustand-demo.pmnd.rs
Tailwind CSS	https://tailwindcss.com/docs
recharts	https://recharts.org
slowapi — rate limiting FastAPI	https://github.com/laurentS/slowapi
MercadoPago Developers (AR)	https://www.mercadopago.com.ar/developers/es
MercadoPago SDK Python	https://github.com/mercadopago/sdk-python
MercadoPago SDK React	https://github.com/mercadopago/sdk-react

B. Patrones aplicados en el proyecto

Patrón	Capa	Descripción
Repository Pattern	Backend	Abstracción del acceso a BD. BaseRepository[T] genérico. Facilita testing con mocks.
Unit of Work	Backend	Gestión de transacciones atómicas. El Service opera dentro del contexto UoW sin gestionar la sesión directamente.
Service Layer	Backend	Lógica de negocio centralizada, stateless. Consume el UoW. Independiente del framework.
Snapshot Pattern	Backend/BD	Precios y nombres de producto inmutables al crear el pedido. Garantiza integridad histórica.
Soft Delete	Backend/BD	deleted_at TIMESTAMPTZ — registros lógicamente eliminados. Nunca DELETE físico en entidades de negocio.
Audit Trail Append-Only	Backend/BD	HistorialEstadoPedido: solo INSERT, nunca UPDATE/DELETE (RN-03). Trazabilidad completa.
State Machine	Backend	Transiciones del pedido validadas en la capa de servicio contra el mapa de transiciones permitidas.
Idempotent Payments	Backend	UUID como idempotency_key enviado a MercadoPago. Evita cobros duplicados por reintentos.

Patrón	Capa	Descripción
Feature-Sliced Design	Frontend	Organización por features con límites de importación claros. Cada feature es autocontenida.
Custom Hooks	Frontend	Encapsulan lógica de TanStack Query en hooks reutilizables por dominio.
Optimistic Updates	Frontend	Actualización inmediata de UI antes de confirmar respuesta del servidor. Rollback en error.
Webhook / IPN	Backend	MercadoPago notifica de forma asíncrona el resultado del pago. Evita polling constante.

— Fin de la Especificación Técnica · Food Store v3.0 —